In this chapter, you'll see:

- How to generate readable documentation from your specs
- How to identify the slowest examples in a suite
- How to run just the specs you care about at any given moment
- How to mark work in progress and come back to it later

CHAPTER 2

From Writing Specs to Running Them

You've installed RSpec and taken it for a test drive. You've written a few specs and gotten a feel for how they're different from test cases in traditional frameworks. You've also seen a few ways to trim repetition from your examples.

In the process, you've applied the following practices:

- Structuring your examples logically into groups
- Writing clear expectations that test at the right level of detail
- Sharing common setup code across specs

RSpec is designed around these habits, but you could learn to apply them to other test frameworks as well. You may be wondering if all that separates RSpec from the crowd is syntax.

In this chapter, we're going to show you that RSpec's usefulness isn't confined to how your specs *look*. It also applies to how they *run*. You're going to learn the following practices that will help you find problems in code more quickly:

- See your specs' output printed as documentation, to help your future self understand the intent of the code when something goes wrong
- Run a specific set of examples, to focus on one slice of your program at a time
- Fix a bug and rerun just the specs that failed last time
- Mark work in progress to remind you to finish something later

The tool that makes these activities possible—and even easy—is RSpec's *spec runner*. It decides which of your specs to run and when to run them. Let's take a look at how to make it sing.